Tara 11-4-26 Conversation 3: Objective/constraint calculator skill

User

I want to build the objective/constraint calculator skill in the system diagram in the 11-4-26 (Tara) folder. The objective will always be to minimize mass of the structure. The constraints will always include a check against the failure criteria using the failure-check skill. There may also be other constraints, including but not limited to dimensions along certain axes (e.g. fixed or minimum length for a boom or maximum thickness for the laminate), minimum allowable stiffness, maximum allowable deflection, minimum allowable buckling load, etc. Create a skill that takes only the output of the OpenSeesPy aggregator skill (when it runs the OpenSeesPy script and produces the results) and uses it to evaluate the mass minimization objective, failure criteria constraints, and any other user-specified constraints (which are provided to the orchestrator in natural language and must be turned into a mathematical formula), and returns that information to the orchestrator.

Claude

Tool: Bash - List the folder and find existing skills

```
cd "D:/Users/tvenkata/projects/ME295_project/Code for Project/11-4-26 (Tara)" && ls -la && git -C .. log --one
  line -5 -- "11-4-26 (Tara)"; ls ../.claude ../../.claude 2>/dev/null; find .. -maxdepth 4 -name SKILL.md 2>/
  dev/null | head -30
```

Result:

```
total 1488
drwxr-xr-x 1 tvenkata 1049089      0 Nov  4 13:21 .
drwxr-xr-x 1 tvenkata 1049089      0 Nov  4 11:52 ..
-rw-r--r-- 1 tvenkata 1049089 125638 Nov  4 13:06 abaqus_plate_tip_load_laminate.py
-rw-r--r-- 1 tvenkata 1049089   1695 Nov  4 13:06 abaqus_plate_tip_load_laminate_instructions.txt
-rw-r--r-- 1 tvenkata 1049089 173026 Nov  4 13:06 abaqus_tape_spring_L024_tip_load.py
-rw-r--r-- 1 tvenkata 1049089   1718 Nov  4 13:06 abaqus_tape_spring_L024_tip_load_instructions.txt
-rw-r--r-- 1 tvenkata 1049089 171008 Nov  4 13:06 abaqus_trac_like_combination.py
-rw-r--r-- 1 tvenkata 1049089   1662 Nov  4 13:06 abaqus_trac_like_combination_instructions.txt
-rw-r--r-- 1 tvenkata 1049089 170788 Nov  4 13:06 abaqus_trac_like_load_control.py
-rw-r--r-- 1 tvenkata 1049089   1676 Nov  4 13:06 abaqus_trac_like_load_control_instructions.txt
-rw-r--r-- 1 tvenkata 1049089    894 Nov  4 11:19 lam_L024_pm45s.json
-rw-r--r-- 1 tvenkata 1049089    404 Nov  4 11:19 plate_tip_load_laminate.json
drwxr-xr-x 1 tvenkata 1049089      0 Nov  4 11:38 plate_tip_load_laminate_code
-rw-r--r-- 1 tvenkata 1049089  64773 Nov  4 11:38 plate_tip_load_laminate_opensees.py
drwxr-xr-x 1 tvenkata 1049089      0 Nov  4 11:41 results
drwxr-xr-x 1 tvenkata 1049089      0 Nov  4 11:38 separate_skills
-rw-r--r-- 1 tvenkata 1049089   7002 Nov  4 13:21 system_diagram.py
-rw-r--r-- 1 tvenkata 1049089 409068 Nov  4 13:21 system_diagram_verification.png
-rw-r--r-- 1 tvenkata 1049089    454 Nov  4 11:19 tape_spring_L024_tip_load.json
drwxr-xr-x 1 tvenkata 1049089      0 Nov  4 11:38 tape_spring_L024_tip_load_code
-rw-r--r-- 1 tvenkata 1049089 189756 Nov  4 11:38 tape_spring_L024_tip_load_opensees.py
-rw-r--r-- 1 tvenkata 1049089    665 Nov  4 11:19 trac_like_combination.json
drwxr-xr-x 1 tvenkata 1049089      0 Nov  4 11:39 trac_like_combination_code
-rw-r--r-- 1 tvenkata 1049089  81110 Nov  4 11:39 trac_like_combination_opensees.py
-rw-r--r-- 1 tvenkata 1049089    643 Nov  4 11:19 trac_like_load_control.json
drwxr-xr-x 1 tvenkata 1049089      0 Nov  4 11:38 trac_like_load_control_code
-rw-r--r-- 1 tvenkata 1049089  80695 Nov  4 11:38 trac_like_load_control_opensees.py
682eae8 System diagram: lamina table adds density and thickness, simpler labels
5d727b4 Abaqus scripts write the ODB extract; run instructions; conversation PDFs
6ad47b8 opensees-to-abaqus skill, results comparator agent, revised system diagram
d73ef25 Added skill to assemble and run OpenSeesPy code generated by layup, mesh, and load/BC skills
../../.claude:
REORGANIZATION_LOG_v2.0.md
agents
agents_v1.0
agents_v2.0
memory.boom_geometry_catalog
memory.layup_catalog
memory.material_property_catalog
... (2 more lines omitted)
```

Tool: Bash - Read system diagram and list skills

```
cd "D:/Users/tvenkata/projects/ME295_project" && ls .claude/skills .claude/skills/* && cat "Code for Project/1
  1-4-26 (Tara)/system_diagram.py" && ls "Code for Project/11-4-26 (Tara)/results" "Code for Project/11-4-26 (
  Tara)/separate_skills" "Code for Project/11-4-26 (Tara)/trac_like_combination_code"
```

Result:

```
.claude/skills/README.md

.claude/skills:
README.md
abaqus-export
failure-check
laminate-properties
opensees-to-abaqus
regression-tests
shell-assemble-run
shell-bc-load
shell-buckling
shell-mesh
shell-visualize

.claude/skills/abaqus-export:
SKILL.md
case_to_abaqus.py

.claude/skills/failure-check:
SKILL.md
failure_check.py

.claude/skills/laminate-properties:
SKILL.md
laminate_properties.py

.claude/skills/opensees-to-abaqus:
SKILL.md
opensees_to_abaqus.py

.claude/skills/regression-tests:
SKILL.md
abaqus_mock.py
last_run.txt
last_run_T11_T12.txt
last_run_full.txt
run_regression.py

.claude/skills/shell-assemble-run:
... (184 more lines omitted)
```

Tool: Bash - Read skills README, aggregator and failure-check docs

```
cd "D:/Users/tvenkata/projects/ME295_project/.claude/skills" && cat README.md && cat shell-assemble-run/SKILL.
  md && cat failure-check/SKILL.md
```

Result:

```
# Thin-shell simulation skills (v2.0)

Five scripts replace the seven v1.0 agents (`.claude/agents_v1.0/`) and the
repeated model code in `Code for Project/`. Each folder has a `SKILL.md`
(instructions) and one Python script. The orchestrator agent
(`.claude/agents_v2.0/ThinShellOrchestrator.md`) runs them in order;
`CompositeLaminaLibrarian` (v2.0) still maintains the material catalog; the
opensees-to-abaqus skill (11/4/26) replaces the `OpenSeesPyToAbaqusPythonGenerator`
agent and turns an assembled OpenSeesPy script into an Abaqus/CAE script.

```
catalog (Pa, m) --> laminate-properties --> shell-mesh --> shell-bc-load --> results .npz --+--> shell-visuali
  ze (PNG / window / .vtp)
 or E1 E2 ...       (ply, layup, ABD,       (cross-section,  (supports, loads,                |
                     OpenSees sections)      mesh, elements)  solve, strains)                 +--> failure-ch
  eck (ply stresses, Hashin)
case JSON ----------------------------------------------------------------------> abaqus-export (Abaqus/CAE script
   + OpenSees reference)
laminate-properties / shell-mesh / shell-bc-load --emit-code pieces --> shell-assemble-run (ONE standalone Ope
  nSeesPy script, run) --> results .npz
                                                                                  |
                                                                                  +--> opensees-to-abaqus (Ab
  aqus/CAE script via abaqus-export, model checked vs the script)
results .npz + Abaqus ODB (odb_extract.py --> <job>_odb.npz) --> OpenSeesAbaqusResultsComparator agent
   (.claude/verification/compare_results.py: outputs only, no model-building code; plots + comparison tables)
```

Skill	Replaces (v1.0 agents / code)
laminate-properties	PlyPropertyGenerator, LayupFromPlyGenerator, TheoreticalABDGenerator, ABDFromOpenSeesP
yLayup; ply_properties.py, layup.py, theoretical_abd.py, abd_from_opensees.py, ply_materials_*.py, layup_*.p
y
```

```
| shell-mesh | MeshBCLoadGenerator (geometry/mesh part); mesh_bc_load.StripMesh, flat_plate_al.py |
| shell-bc-load | MeshBCLoadGenerator (supports/loads/solve); mesh_bc_load.py, plate_tip_load.py, plate_large_
  deflection.py, elastica.py |
| shell-visualize | FEResultVisualizer; fe_result_viewer.py, shell_viewer.py |
| failure-check | new (ToDo items 11-12, Hashin 2D of AIAA 2025-0188); top_surface_stresses() variants |
| abaqus-export | OpenSeesPyToAbaqusPythonGenerator's hand translation; abaqus_tip_load_laminate.py template |
| shell-assemble-run | new: joins the `--emit-code` pieces of laminate-properties, shell-mesh and shell-bc-loa
  d into one continuous OpenSeesPy script and runs it |
... (180 more lines omitted)
```

Tool: Bash - Inspect results npz contents and failure_check API

```
cd "D:/Users/tvenkata/projects/ME295_project" && PY=/d/Users/tvenkata/miniforge3/envs/shells/python.exe; $PY -
  c "
import numpy as np
for f in ['trac_like_combination','tape_spring_L024_tip_load','plate_tip_load_laminate','trac_like_load_contro
  l']:
    d=np.load('Code for Project/11-4-26 (Tara)/results/'+f+'.npz',allow_pickle=True)
    print(f)
    for k in d.files:
        v=d[k]; print('  ',k,v.shape,v.dtype, (str(v)[:300] if v.dtype.kind in 'USO' else ''))
"; grep -n "^def \|^class \|^    def " .claude/skills/failure-check/failure_check.py | head -60
```

Result:

```
trac_like_combination
   points (493, 3) float64
   node_tags (493,) int64
   disp (493, 6) float64
   fixed (493,) int8
   faces (440, 4) int64
   frames (440, 3, 3) float64
   strains (440, 4, 8) float64
   history (4, 3) float64
   laminate () <U697 {"kind": "laminate", "name": "", "n_sub": 4, "merge": true, "G_out": null, "materials": [
  {"E1": 80080.0, "E2": 6670.0, "E3": 6670.0, "nu12": 0.344, "nu13": 0.344, "nu23": 0.33, "G12": 2930.0, "G13"
  : 2930.0, "G23": 2507.518796992481, "rho": 0.0, "nu23_assumed": false, "nu21": 0.028652347652347653, "n
   meta () <U954 {"title": "trac_like_combination", "summary": "200 x 55 mm shell, segments (3 pieces); 40 x 1
  1 = 440 quads of 5 x 5 mm, 492 nodes", "nx": 40, "ny": 11, "L": 200, "b": 55.0, "element": "ASDShellQ4", "ri
  gid": {"x0": 493}, "case": {"name": "trac_like_combination", "laminate": {"json": "D:\\Users\\tvenk
tape_spring_L024_tip_load
   points (2122, 3) float64
   node_tags (2122,) int64
   disp (2122, 6) float64
   fixed (2122,) int8
   faces (2000, 4) int64
   frames (2000, 3, 3) float64
   strains (2000, 4, 8) float64
   history (2, 1) float64
   laminate () <U693 {"kind": "laminate", "name": "", "n_sub": 4, "merge": true, "G_out": null, "materials": [
  {"E1": 80080.0, "E2": 6670.0, "E3": 6670.0, "nu12": 0.344, "nu13": 0.344, "nu23": 0.33, "G12": 2930.0, "G13"
  : 2930.0, "G23": 2507.518796992481, "rho": 0.0, "nu23_assumed": false, "nu21": 0.028652347652347653, "n
   meta () <U679 {"title": "tape_spring_L024_tip_load", "summary": "500 x 100 mm shell, arc R = 50 mm (114.6 d
  eg); 100 x 20 = 2000 quads of 5 x 5 mm, 2121 nodes", "nx": 100, "ny": 20, "L": 500, "b": 100.0, "element": "
  ASDShellQ4", "rigid": {"xL": 2122}, "case": {"name": "tape_spring_L024_tip_load", "laminate": {"cat
plate_tip_load_laminate
   points (1111, 3) float64
   node_tags (1111,) int64
   disp (1111, 6) float64
   fixed (1111,) int8
   faces (1000, 4) int64
   frames (1000, 3, 3) float64
   strains (1000, 4, 8) float64
   history (2, 1) float64
   laminate () <U674 {"kind": "laminate", "name": "", "n_sub": 4, "merge": true, "G_out": null, "materials": [
... (42 more lines omitted)
```

Tool: Bash - Dump laminate/meta JSON and failure_check header

```
cd "D:/Users/tvenkata/projects/ME295_project" && PY=/d/Users/tvenkata/miniforge3/envs/shells/python.exe; $PY -
  c "
import numpy as np,json
for f in ['trac_like_combination','tape_spring_L024_tip_load','trac_like_load_control']:
    d=np.load('Code for Project/11-4-26 (Tara)/results/'+f+'.npz',allow_pickle=True)
    print(json.dumps(json.loads(str(d['laminate'])),indent=0)); print(json.dumps(json.loads(str(d['meta'])),in
  dent=0)); print(d['history'])
"; sed -n 1,82p .claude/skills/failure-check/failure_check.py
```

Result:

```
{
"kind": "laminate",
"name": "",
"n_sub": 4,
"merge": true,
```

```
"G_out": null,
"materials": [
{
"E1": 80080.0,
"E2": 6670.0,
"E3": 6670.0,
"nu12": 0.344,
"nu13": 0.344,
"nu23": 0.33,
"G12": 2930.0,
"G13": 2930.0,
"G23": 2507.518796992481,
"rho": 0.0,
"nu23_assumed": false,
"nu21": 0.028652347652347653,
"nu31": 0.028652347652347653,
"nu32": 0.32999999999999996,
"Xt": 1063.48,
"Xc": 662.6899999999999,
"Yt": 65.99,
"Yc": 144.54,
"S": 126.75999999999999,
"t_ply": 0.08,
"name": "L024 carbon, unspecified / epoxy (lenticular boom CFRP)"
}
],
"plies": [
{
"theta": 45.0,
"t": 0.08,
"mat": 0
},
{
"theta": -45.0,
"t": 0.08,
... (490 more lines omitted)
```

Still surveying: the results files carry `rho = 0`, so I'm checking where density lives (the lamina catalog) before designing the mass calculation.

Tool: Bash - Read failure_check internals and catalog density info

```
cd "D:/Users/tvenkata/projects/ME295_project" && sed -n 83,520p .claude/skills/failure-check/failure_check.py;
   ls .claude/memory.material_property_catalog; head -40 .claude/memory.material_property_catalog/UD_lamina_ca
  talog_SI.txt; grep -n "rho\|density" .claude/skills/laminate-properties/laminate_properties.py | head -30
```

Result:

```
def hashin_modes(s1, s2, t12, Xt, Xc, Yt, Yc, S, S23=None):
    """(a, b) of each mode, F_mode(R) = a R^2 + b R for stresses scaled by R
    (inactive modes a = b = 0). Arrays of shape (4,) + s1.shape."""
    S23 = Yc / 2 if S23 is None else S23
    s1, s2, t12 = np.broadcast_arrays(*(np.asarray(x, float) for x in (s1, s2, t12)))
    ft, mt = s1 >= 0, s2 >= 0
    shear = (t12 / S)**2
    a = np.stack([np.where(ft, (s1 / Xt)**2 + shear, 0.0),
                  np.where(ft, 0.0, (s1 / Xc)**2),
                  np.where(mt, (s2 / Yt)**2 + shear, 0.0),
                  np.where(mt, 0.0, (s2 / (2 * S23))**2 + shear)])
    b = np.zeros_like(a)
    b[3] = np.where(mt, 0.0, ((Yc / (2 * S23))**2 - 1) * s2 / Yc)
    return a, b


def hashin(s1, s2, t12, Xt, Xc, Yt, Yc, S, S23=None):
    """Hashin 2D index F = max over the active modes, and the mode index."""
    a, b = hashin_modes(s1, s2, t12, Xt, Xc, Yt, Yc, S, S23)
    F = a + b
    return F.max(axis=0), F.argmax(axis=0)


def tsai_hill(s1, s2, t12, Xt, Xc, Yt, Yc, S):
    """Plane-stress Tsai-Hill index, element-wise on arrays."""
    X = np.where(s1 >= 0, Xt, Xc)
    Y = np.where(s2 >= 0, Yt, Yc)
    return (s1 / X)**2 - s1 * s2 / X**2 + (s2 / Y)**2 + (t12 / S)**2


def strength_ratio(a, b):
    """Smallest R > 0 with a R^2 + b R = 1 over the modes (inf if none)."""
    with np.errstate(divide='ignore', invalid='ignore'):
```

```
            R = np.where(a > 0, (-b + np.sqrt(b * b + 4 * a)) / (2 * a), np.where(b > 0, 1 / b, np.inf))
        return R.min(axis=0)


    def ply_stresses(strains, lam, at='gauss'):
        """strains: (nf, ng, 8) from shell-bc-load. Returns (sig12, sigxy, z):
        stresses (nf, ng', n_ply, 3 z-points, 3) in ply axes [s1, s2, t12] and in
    ... (424 more lines omitted)
```

Tool: Bash - Check shell_bc_load results, history, loads and catalog row

```
    cd "D:/Users/tvenkata/projects/ME295_project/.claude/skills" && grep -n "^def \|^class \|history\|np.savez\|re
      port_node\|'rigid'" shell-bc-load/shell_bc_load.py | head -80; cat "../../Code for Project/11-4-26 (Tara)/la
      m_L024_pm45s.json"; grep -n "^L024\|^# Columns" ../memory.material_property_catalog/UD_lamina_catalog_SI.txt
      ; sed -n 80,120p laminate-properties/laminate_properties.py
```

Result:

```
    62:def fix_edge(mesh, edge, dofs=(1, 1, 1, 1, 1, 1)):
    69:def fix_node(node, dofs):
    73:def rigid_end(mesh, edge, master_tag=None):
    85:    if not hasattr(mesh, 'rigid'):
    93:def load_pattern(tag=1, series_tag=1):
    98:def _pattern(tag):
    103:def edge_load(mesh, edge, force, moment=(0.0, 0.0, 0.0), pattern_tag=1, scale=1.0):
    124:def point_load(node, force, moment=(0.0, 0.0, 0.0), pattern_tag=1, scale=1.0):
    129:def pressure_load(mesh, p, pattern_tag=1, scale=1.0):
    145:def resolve_node(mesh, ref):
    147:    {'i': .., 'j': ..} | {'rigid': 'xL'} | 'tip' (x = L, mid-section)."""
    152:    if 'rigid' in ref:
    153:        return mesh.rigid[ref['rigid']]
    159:def apply_supports(mesh, supports):
    176:def apply_loads(mesh, loads, pattern_tag=1):
    193:def static_setup(tol=1e-6, max_iter=40):
    201:def linear_solve():
    214:def displacement_control(node, dof, step, n_steps=None, stop=None, max_cuts=6, record=None,
    223:    history = [[0.0, *[0.0] * len(record()), 0]]
    225:    while (n_steps is None or k < n_steps) and not (stop and stop(history[-1][0])):
    235:                print(f'  stopped: no convergence beyond load factor {history[-1][0]:.4g}')
    237:        history.append([ops.getLoadFactor(pattern_tag), *record(), ops.testIter()])
    240:            print(f'  step {k:3d}: load factor {history[-1][0]:.5g}, {history[-1][-1]} iterations', flush=
      True)
    242:    return np.array(history)
    245:def load_control(n_steps, max_factor=1.0, record=None, max_cuts=6, verbose=True, tol=1e-6):
    250:    history = [[0.0, *[0.0] * len(record()), 0]]
    265:        history.append([lam, *record(), ops.testIter()])
    269:    return np.array(history)
    272:def modal(n=1):
    279:def facet_strains(mesh):
    298:def collect_results(mesh, history=None, title='', case=None):
    307:                element=getattr(mesh, 'element', ''), rigid=getattr(mesh, 'rigid', {}), case=case)
    311:                history=np.zeros((0, 0)) if history is None else np.asarray(history),
    315:def save_results(path, res):
    317:    np.savez_compressed(path, **res)
    321:def load_results(path):
    330:def run_case(case, out_dir=None, verbose=True):
    385:history = np.array([[0.0], [1.0]])
    400:history = [[0.0, 0.0, 0]]            # [load factor, control displacement, iterations]
    ... (121 more lines omitted)
```

Tool: Bash - Read load application and results code

```
    cd "D:/Users/tvenkata/projects/ME295_project/.claude/skills" && sed -n 60,80p laminate-properties/laminate_pro
      perties.py; sed -n 100,200p shell-bc-load/shell_bc_load.py; sed -n 296,385p shell-bc-load/shell_bc_load.py;
      sed -n 470,500p shell-bc-load/shell_bc_load.py
```

Result:

```
    STRENGTHS = ('Xt', 'Xc', 'Yt', 'Yc', 'S')


    # --- catalog ------------------------------------------------------------

    def _cell(c):
        """(float or None, is_estimate) of a catalog cell: NA, 1.2e9, 0.45*."""
        if c == 'NA':
            return None, False
        return float(c.rstrip('*')), c.endswith('*')


    def read_catalog(path=CATALOG):
        """{ID: row dict} of the SI catalog. Columns are separated by 2+ spaces."""
        rows, header = {}, None
        for line in Path(path).read_text(encoding='utf-8').splitlines():
            if not line.strip() or line.startswith('#'):
```

```
            continue
        cells = [c.strip() for c in re.split(r'\t| {2,}', line.strip())]
        if header is None:
            header = cells
        load_pattern(tag, tag)


def edge_load(mesh, edge, force, moment=(0.0, 0.0, 0.0), pattern_tag=1, scale=1.0):
    """Total force [N] (and moment [N mm]) spread uniformly along an edge as
    consistent nodal loads (end nodes half shares). Fixed global direction.
    Uniform per node spacing; for non-uniform spacing use tributary lengths
    (done automatically when the edge spacing varies)."""
    nodes = mesh.edge_nodes(edge)
    _pattern(pattern_tag)
    xyz = np.array([ops.nodeCoord(n) for n in nodes])
    seg = np.linalg.norm(np.diff(xyz, axis=0), axis=1)
    if np.ptp(seg) < 1e-9 * seg.max():
        n = len(nodes) - 1
        w = np.array([(0.5 if k in (0, n) else 1.0) / n for k in range(n + 1)])
    else:
        w = np.zeros(len(nodes))
        w[:-1] += seg / 2
        w[1:] += seg / 2
... (204 more lines omitted)
```

Tool: Bash - Check orchestrator agent for objective/constraint references

```
cd "D:/Users/tvenkata/projects/ME295_project/.claude" && ls agents agents_v2.0; grep -n -i "constraint\|object
  ive\|failure-check\|mass" agents_v2.0/*.md agents/*.md | head -30; grep -n "def node_tag\|def edge_nodes\|de
  f node_at" skills/shell-mesh/shell_mesh.py
```

Result:

```
agents:
CompositeLaminaLibrarian.md
OpenSeesAbaqusResultsComparator.md
ThinShellOrchestrator.md

agents_v2.0:
CompositeLaminaLibrarian.md
OpenSeesAbaqusResultsComparator.md
ThinShellOrchestrator.md
agents_v2.0/CompositeLaminaLibrarian.md:3:description: Finds, checks and catalogs measured unidirectional (UD)
   lamina properties of fiber reinforced polymer composites (E1, E2, G12, nu12, nu23, G23, Xc, Xt, Yc, Yt, S,
  ply thickness, density) from referenced literature, and maintains the lamina catalog .txt files (readable GP
  a/MPa/mm version and SI version). Use when the user wants lamina properties looked up, added to, checked or
  exported from the catalog, e.g. to feed the laminate-properties skill or the failure-check skill.
agents_v2.0/CompositeLaminaLibrarian.md:111:- Usable by the failure-check skill (Hashin 2D / Tsai-Hill): Xt, X
  c, Yt, Yc, S all present.
agents_v2.0/ThinShellOrchestrator.md:3:description: Sets up, runs, checks and reports OpenSeesPy simulations o
  f thin composite (or isotropic) shells in the ME295 project - flat plates, tape springs, TRAC/omega-type ext
  ruded sections - by running the project skills in order - laminate-properties (catalog lamina -> OpenSees la
  minate), shell-mesh (cross-section -> meshed component), shell-bc-load (supports, load combination, solve),
  shell-visualize (curvature and stress pictures) and failure-check (ply stresses and Hashin 2D index: how clo
  se to failure), and on request shell-assemble-run (one standalone OpenSeesPy script) and opensees-to-abaqus
  (equivalent Abaqus/CAE script of that model). Use for any shell simulation request, from material choice to
  failure check.
agents_v2.0/ThinShellOrchestrator.md:88:python failure-check/failure_check.py <dir>/results/<case>.npz
        # Hashin 2D (default)
agents/CompositeLaminaLibrarian.md:3:description: Finds, checks and catalogs measured unidirectional (UD) lami
  na properties of fiber reinforced polymer composites (E1, E2, G12, nu12, nu23, G23, Xc, Xt, Yc, Yt, S, ply t
  hickness, density) from referenced literature, and maintains the lamina catalog .txt files (readable GPa/MPa
  /mm version and SI version). Use when the user wants lamina properties looked up, added to, checked or expor
  ted from the catalog, e.g. to feed the laminate-properties skill or the failure-check skill.
agents/CompositeLaminaLibrarian.md:111:- Usable by the failure-check skill (Hashin 2D / Tsai-Hill): Xt, Xc, Yt
  , Yc, S all present.
agents/ThinShellOrchestrator.md:3:description: Sets up, runs, checks and reports OpenSeesPy simulations of thi
  n composite (or isotropic) shells in the ME295 project - flat plates, tape springs, TRAC/omega-type extruded
   sections - by running the project skills in order - laminate-properties (catalog lamina -> OpenSees laminat
  e), shell-mesh (cross-section -> meshed component), shell-bc-load (supports, load combination, solve), shell
  -visualize (curvature and stress pictures) and failure-check (ply stresses and Hashin 2D index: how close to
   failure), and on request shell-assemble-run (one standalone OpenSeesPy script) and opensees-to-abaqus (equi
  valent Abaqus/CAE script of that model). Use for any shell simulation request, from material choice to failu
... (7 more lines omitted)
```

Tool: Bash - Read orchestrator agent structure

```
cd "D:/Users/tvenkata/projects/ME295_project/.claude" && sed -n 60,130p agents/ThinShellOrchestrator.md; diff
  agents/ThinShellOrchestrator.md agents_v2.0/ThinShellOrchestrator.md && echo SAME; grep -n "^#\|^- \*\*" age
  nts/ThinShellOrchestrator.md | head -40
```

Result:

```
## 1. Collect the inputs (ask for what is missing; never invent values)
1. **Material**: catalog ID (e.g. L024), or E1, E2, nu12, G12 [nu23] and, for a
```

```
   failure check, Xt, Xc, Yt, Yc, S; or isotropic E, nu (and yield stress).
2. **Layup**: a layup-catalog ID (Y###), or angles bottom (-z) to top (+z);
   and ply thickness (default: the catalog t_ply, 0.045 mm). If the order is not stated, take the first ply as
   the
   bottom one and say so. Angles from the extrusion axis x, CCW about the
   shell normal.
3. **Geometry**: length L; cross-section (flat width b; arc R and arc length b;
   lines/arcs chain; or (y, z) points); element size (default 5 mm for
   nonlinear runs).
4. **Supports**: edges (`x0 xL y0 yb`) and DOFs, or a rigid fixture (rigid end).
5. **Loads**: each load's location, global direction and magnitude, and
   whether it is one combination (applied together) or several cases.
6. **Analysis**: linear, or large deflection (control node/DOF and target
   displacement or load factor).

## 2. Run
Write a case file in a new dated folder `Code for Project/MM-DD-YY (Name)/`
(see `.claude/skills/shell-bc-load/SKILL.md` and `examples/`) and run, from
`.claude/skills`:
```
python laminate-properties/laminate_properties.py <material/layup args> --check --save-json <dir>/lam.json
python shell-bc-load/shell_bc_load.py <dir>/<case>.json --out <dir>/results
or, for one standalone OpenSeesPy script of the whole model (same .npz): the three
skills write their code pieces (--emit-code) and shell-assemble-run joins and runs them
python shell-assemble-run/assemble_run.py <dir>/<case>.json [--laminate <dir>/lam.json] [--verify]
and, for an Abaqus/CAE version of that script (run elsewhere; checked here on a stand-in API):
python opensees-to-abaqus/opensees_to_abaqus.py <dir>/<case>_opensees.py [--element S4R|S4]
python failure-check/failure_check.py <dir>/results/<case>.npz   # Hashin 2D (default)
python shell-visualize/shell_visualize.py <dir>/results/<case>.npz --png <dir>/results/<case>.png --no-gui
```
For things a case file cannot express (parameter sweeps, several load
cases, convergence studies, custom references) write a short driver script
in the same folder that imports the skill modules (see each SKILL.md
"Python" section) and keeps the run reproducible. Run anything longer than
~2 minutes in the background.

## 3. Check before reporting
- laminate-properties `--check`: A, B ~1e-15, D low by ~1/(layers)^2; else stop.
... (41 more lines omitted)
```

Tool: Bash - Read Laminate class and catalog header

```
cd "D:/Users/tvenkata/projects/ME295_project/.claude" && sed -n 225,345p skills/laminate-properties/laminate_p
  roperties.py; grep -n "^ID" memory.material_property_catalog/UD_lamina_catalog_SI.txt; grep -c "^L0" memory.
  material_property_catalog/UD_lamina_catalog_SI.txt; grep "NA *http\|  NA  " memory.material_property_catalog
  /UD_lamina_catalog_SI.txt | awk '{print $1}' | head; ls skills/shell-bc-load/examples
```

Result:

```
# --- layup notation -----------------------------------------------------------

def expand_layup(text):
    """'[0/±45/90]s' -> [0, 45, -45, 90, 90, -45, 45, 0]. Supports ± or +-
    (+theta then -theta), -+ / ■, item repeats 0_2, group repeats [..]2 or
    [..]_2, symmetry s (also 2s), and plain lists '0 45 -45 90'."""
    t = text.replace(' ', '')
    m = re.fullmatch(r'\[(.*)\]_?(\d*)(s?)', t, flags=re.I)
    if not m:
        return [float(a) for a in re.split(r'[,/\s]+', text.strip()) if a]
    body, rep, sym = m.groups()
    out = []
    for item in body.split('/'):
        im = re.fullmatch(r'(±|\+-|■|-\+)?([+-]?\d+(?:\.\d*)?)(?:_(\d+))?', item)
        if not im:
            raise ValueError(f'cannot read ply {item!r} in {text!r}')
        pm, ang, n = im.groups()
        a = float(ang)
        unit = {None: [a], '±': [a, -a], '+-': [a, -a], '■': [-a, a], '-+': [-a, a]}[pm]
        out += unit * int(n or 1)
    out *= int(rep or 1)
    if sym:
        out += out[::-1]
    return out


# --- laminate -----------------------------------------------------------------

class Laminate:
    """Plies bottom (-z) to top (+z): dicts theta [deg], t [mm], mat (index
    into materials), z0, z1 (from the mid-surface). kind 'laminate' (UD plies)
    or 'isotropic' (one ElasticMembranePlateSection, E nu t)."""

    def __init__(self, materials, plies, n_sub=None, merge=True, G_out=None, kind='laminate', name=''):
```

```
        self.materials, self.kind, self.name = list(materials), kind, name
        self.merge, self.G_out = merge, G_out
        h, z = sum(p['t'] for p in plies), None
        z = -h / 2
        self.plies = []
        for p in plies:
... (88 more lines omitted)
```

Writing the script now.

Tool: Write

```
D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\objective_constraints.py
"""
Objective and constraints of one design, from the results .npz of a solved
OpenSeesPy shell (written by the script that shell-assemble-run assembles
and runs; shell_bc_load.py writes the same file). Nothing else is read: the
laminate, the case (geometry, supports, loads, analysis) and the mesh are
the ones stored in the .npz.

Objective (always): minimize the mass of the structure,
    m = sum over facets of (facet area) x (areal density of its laminate),
areal density = sum over plies of rho t. Facet areas are the undeformed quad
areas 0.5 |(X2 - X0) x (X3 - X1)|. TRAC/OM/CO web facets carry the doubled
(bonded) laminate, found as the failure-check skill finds them. Rigid end
fixtures have no mass. Density: the laminate's rho if > 0, else the lamina
catalog row named by the material (catalog_material names it 'L024 ...'),
else --rho [kg/m^3]; without any, the mass is not evaluated.

Constraints (always): no failure, F_governing < 1, where F_governing is the
larger of the simulation Hashin 2D index (or Tsai-Hill) and the coiling
(flattening) index, computed with the failure-check skill's functions.

Other constraints: formulas written by the orchestrator from the user's
words, each
    {"name": .., "text": <user's words>, "expr": <formula>, "op": "<=" | "<" | ">=" | ">" | "==",
     "limit": <number or formula>, "tol": <for ==>, "units": ..}
in a JSON file (--constraints, a list or {"constraints": [...]}), or inline
--con "name: expr <= limit". Formulas use the quantity names printed by
--list (N, mm, MPa, kg), numbers, + - * / ** %, and abs min max sqrt
log exp sin cos tan radians degrees pi. Each constraint is reported in
the normalized form g <= 0:
    <=, < : g = value - limit         >=, > : g = limit - value
    ==    : g = |value - limit| - tol (tol default 1e-6 max(1, |limit|))
with g_rel = g / |limit| (g if limit = 0). A constraint whose formula uses
a quantity that this file cannot give (e.g. a buckling load: the
assembled script runs a static analysis) is "not evaluated" unless the
value is passed with --value NAME=VALUE (e.g. from the shell-buckling skill).

Usage:
  python objective_constraints.py results.npz [--constraints c.json] [--con "name: expr op limit" ...]
          [--value NAME=VALUE ...] [--rho KG_M3] [--json out.json] [--list]
          [failure-check options: --criterion --S23 --at --Xt --Xc --Yt --Yc --S --yield
           --boom --web S0 S1 --web-stack --flatten-radius]
Units: N, mm, MPa; mass in kg.
"""
import argparse
import ast
import json
import math
import re
import sys
from pathlib import Path

import numpy as np

_SKILLS = Path(__file__).resolve().parents[1]
for _d in ('laminate-properties', 'shell-bc-load', 'failure-check'):
    sys.path.insert(0, str(_SKILLS / _d))
import laminate_properties as lp   # noqa: E402
import failure_check as fc         # noqa: E402

... (407 more lines omitted)
```

Result:

```
File created successfully at: D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\ob
  jective_constraints.py (file state is current in your context — no need to Read it back)
```

Tool: Bash - Run new skill on test results and cross-check failure-check

```
cd "D:/Users/tvenkata/projects/ME295_project/.claude/skills/objective-constraints" && python - <<'EOF'
p='objective_constraints.py'; s=open(p,encoding='utf-8').read()
```

```
s=s.replace("    F_sim, crit = 0.0, None\n","    F_sim, R_min, crit = 0.0, math.inf, None\n")
s=s.replace("        R_min = float(R.min()) if label == groups[0][0] else min(R_min, float(R.min()))\n","
     R_min = min(R_min, float(R.min()))\n")
open(p,'w',encoding='utf-8').write(s)
EOF
PY=/d/Users/tvenkata/miniforge3/envs/shells/python.exe; R="../../../Code for Project/11-4-26 (Tara)/results"
$PY objective_constraints.py "$R/trac_like_combination.npz" --list
$PY objective_constraints.py "$R/tape_spring_L024_tip_load.npz" --con "tip deflection: abs(tip_uz) <= L/100" -
  -con "length: L == 500" --con "max thickness: h <= 0.5" --con "stiffness: abs(force_z)/abs(tip_uz) >= 1" --c
  on "buckling: buckling_load >= 50" --json "$TEMP/ts.json"
cd ../failure-check && $PY failure_check.py "$R/tape_spring_L024_tip_load.npz" | tail -3; $PY failure_check.py
   "$R/trac_like_combination.npz" | grep -i "GOVERN\|MAX F"
```

Result:

```
/usr/bin/bash: line 1: python: command not found
Objective / constraints: trac_like_combination  [[45/-45/-45/45] h = 0.32 mm]
  mass                   0.00562534 kg      mass of the shell (web facets with the doubled laminate)
  mass_g                    5.62534 g       mass in grams
  mass_flange            0.00562534 kg      mass of the flange (single-laminate) facets
  mass_web                        0 kg      mass of the web (doubled-laminate) facets
  areal_density               0.512 kg/m^2  flange laminate mass per area
  mass_per_length         0.0281267 kg/m    mass / L
  L                             200 mm      length along x
  b                              55 mm      cross-section arc length (flat width)
  section_width             38.7095 mm      cross-section extent along y
  section_height            27.9283 mm      cross-section extent along z
  radius_min                    20 mm       smallest arc radius of the section
  radius_max                    20 mm       largest arc radius of the section
  area                       10987 mm^2     shell mid-surface area
  h                           0.32 mm       flange laminate thickness
  n_plies                        4          plies in the flange laminate
  h_web                        n/a mm       no web
  h_max                       0.32 mm       largest laminate thickness anywhere
  tip_ux                -0.0351175 mm       tip ux (mean of the 12 edge nodes at xL)
  tip_uy                  0.331763 mm       tip uy (mean of the 12 edge nodes at xL)
  tip_uz                  -1.05429 mm       tip uz (mean of the 12 edge nodes at xL)
  tip_rx                0.00269978 rad      tip rx (mean of the 12 edge nodes at xL)
  tip_ry                  0.120307 rad      tip ry (mean of the 12 edge nodes at xL)
  tip_rz                 0.0135462 rad      tip rz (mean of the 12 edge nodes at xL)
  tip_u                   1.10581 mm        tip translation magnitude
  tip_rot                0.121097 rad       tip rotation magnitude (small-rotation vector)
  max_u                   1.94131 mm        largest node translation magnitude
  max_abs_ux             0.221412 mm        largest |ux| over all nodes
  max_abs_uy              1.06925 mm        largest |uy| over all nodes
  max_abs_uz              1.92864 mm        largest |uz| over all nodes
  max_rot                 0.34028 rad       largest node rotation magnitude
  force_x                       0 N         total applied force x at the final load factor
  force_y                -1.30943 N         total applied force y at the final load factor
  force_z                -0.958244 N        total applied force z at the final load factor
  couple_x                      0 N mm      applied couples x at the final load factor (not r x F)
  couple_y                 106.09 N mm      applied couples y at the final load factor (not r x F)
  couple_z                      0 N mm      applied couples z at the final load factor (not r x F)
  load_factor             21.218           final load factor (displacement_control)
  load_factor_peak        21.218           largest |load factor| along the path
... (50 more lines omitted)
```

Tool: Edit

```
D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\objective_constraints.py
--- old
    F_sim, crit = 0.0, None
+++ new
    F_sim, R_min, crit = 0.0, math.inf, None
```

Tool: Edit

```
D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\objective_constraints.py
--- old
        R_min = float(R.min()) if label == groups[0][0] else min(R_min, float(R.min()))
+++ new
        R_min = min(R_min, float(R.min()))
```

Tool: Edit

```
D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\objective_constraints.py
--- old
    """The free end, 'x0' or 'xL': the end with no fixed node (grid or rigid
    master); xL if both or neither are free. Returns (edge, node indices of
    the displacement reported, how)."""
    L, meta = res['meta']['L'], res['meta']
    idx = {t: k for k, t in enumerate(res['node_tags'])}
    rigid = {e: idx[t] for e, t in meta.get('rigid', {}).items() if t in idx}
    fixed = {}
    for e, x in (('x0', 0.0), ('xL', L)):
```

```
            nodes = list(end_nodes(res, x)) + ([rigid[e]] if e in rigid else [])
            fixed[e] = bool(res['fixed'][nodes].any())
+++ new
    """The free end, 'x0' or 'xL': an end is supported if its rigid master is
    fixed or more than half of its edge nodes are (a support along a y edge
    fixes only the corner node); the tip is the unsupported end, xL if both
    or neither are. Returns (edge, node indices of the displacement reported, how)."""
    L, meta = res['meta']['L'], res['meta']
    idx = {t: k for k, t in enumerate(res['node_tags'])}
    rigid = {e: idx[t] for e, t in meta.get('rigid', {}).items() if t in idx}
    fixed = {}
    for e, x in (('x0', 0.0), ('xL', L)):
        fixed[e] = bool(res['fixed'][rigid[e]]) if e in rigid else False
        fixed[e] |= res['fixed'][end_nodes(res, x)].mean() > 0.5
```

Tool: Edit

```
D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\objective_constraints.py
--- old
        notes.append(f'critical point: {fq["critical"]}')
+++ new
        notes.append(f'critical point of the simulation: {fq["critical"]}')
```

Result:

```
The file D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\objective_constraints.p
  y has been updated successfully. (file state is current in your context — no need to Read it back)
```

Result:

```
The file D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\objective_constraints.p
  y has been updated successfully. (file state is current in your context — no need to Read it back)
```

Result:

```
The file D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\objective_constraints.p
  y has been updated successfully. (file state is current in your context — no need to Read it back)
```

Result:

```
The file D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\objective_constraints.p
  y has been updated successfully. (file state is current in your context — no need to Read it back)
```

Tool: Bash - Test JSON constraints, web, missing data and safety

```
cd "D:/Users/tvenkata/projects/ME295_project/.claude/skills/objective-constraints" && PY=/d/Users/tvenkata/min
  iforge3/envs/shells/python.exe; R="../../../Code for Project/11-4-26 (Tara)/results"; S="$TEMP/oc_test"; mkd
  ir -p "$S"
cat > "$S/c.json" <<'EOF'
{"constraints": [
 {"name": "tip deflection", "text": "the tip may not deflect more than 2 mm", "expr": "abs(tip_uz)", "op": "<=
  ", "limit": 2, "units": "mm"},
 {"name": "boom length", "text": "boom at least 150 mm long", "expr": "L", "op": ">=", "limit": 150, "units":
  "mm"},
 {"name": "buckling", "text": "buckling load at least 3 times the applied tip load", "expr": "buckling_load",
  "op": ">=", "limit": "3*abs(force_z)"},
 {"name": "evil", "expr": "__import__('os').system('echo hi')", "op": "<=", "limit": 0},
 {"name": "typo", "expr": "tipp_uz", "op": "<=", "limit": 0}
]}
EOF
echo "=== trac load control, web 0..15 and 40..55, buckling given"; $PY objective_constraints.py "$R/trac_like
  _load_control.npz" --constraints "$S/c.json" --web 0 15 --web 40 55 --value buckling_load=5 --json "$S/t.jso
  n"
echo "=== plate: no strengths, no density"; $PY objective_constraints.py "$R/plate_tip_load_laminate.npz" --co
  n "abs(tip_uz) <= 100" --json "$S/p.json"
echo "=== plate with rho and strengths"; $PY objective_constraints.py "$R/plate_tip_load_laminate.npz" --rho 1
  550 --Xt 1500 --Xc 1200 --Yt 50 --Yc 200 --S 70 --json "$S/p2.json" | head -4
echo "=== failure-check with same web"; cd ../failure-check && $PY failure_check.py "$R/trac_like_load_control
  .npz" --web 0 15 --web 40 55 | grep "GOVERN\|web facets"
```

Result:

```
=== trac load control, web 0..15 and 40..55, buckling given
Objective / constraints: trac_like_load_control  [[45/-45/-45/45] h = 0.32 mm]
OBJECTIVE  minimize mass: 0.00869734 kg (8.697 g)
CONSTRAINTS (g <= 0 is satisfied; g_rel = g / |limit|)
  [satisfied    ] no failure: F_governing < 1.0  value 0.335368, limit 1, g -0.664632, g_rel -0.664632
  [satisfied    ] tip deflection: abs(tip_uz) <= 2  value 0.0810722, limit 2, g -1.91893, g_rel -0.959464
  [satisfied    ] boom length: L >= 150  value 200, limit 150, g -50, g_rel -0.333333
  [satisfied    ] buckling: buckling_load >= 3*abs(force_z)  value 5, limit 0.135486, g -4.86451, g_rel -35.90
  43
  [not evaluated] evil: __import__('os').system('echo hi') <= 0  -- not evaluated: unknown quantity __import__
   (see --list; pass values from other skills with --value NAME=VALUE)
  [not evaluated] typo: tipp_uz <= 0  -- not evaluated: unknown quantity tipp_uz (see --list; pass values from
   other skills with --value NAME=VALUE)
FEASIBLE: unknown (constraints not evaluated)
  note: web facets: 240 of 440 (given with --web; s = 0..15, 40..55 mm)
  note: catalog L024 rho = 1600 kg/m^3
```

```
  note: tip = free end xL: mean of the 12 edge nodes at xL
  note: S23 = Yc/2 (default)
  note: coiling: flange flattened from r = 20 mm (smallest arc radius of the case cross-section), F = 0.3354 (
  matrix tension)
  note: critical point of the simulation: {'group': 'web', 'facet': 119, 'i': 39, 'j': 2, 'ply': 1, 'theta': -
  45.0, 'at': 'bottom', 'mode': 'fibre compression', 's1_s2_t12': [-63.40618225356095, -1.1260305051203188, -1
  .9184011671390837]}
saved D:\Users\tvenkata\AppData\Local\Temp\1\oc_test\t.json
=== plate: no strengths, no density
Objective / constraints: plate_tip_load_laminate  [[0/0/90/90/90/90/0/0] h = 8 mm]
OBJECTIVE  minimize mass: n/a kg  -- density unknown: give --rho [kg/m^3]
CONSTRAINTS (g <= 0 is satisfied; g_rel = g / |limit|)
  [not evaluated] no failure: F_governing < 1.0  -- not evaluated: F_governing not available (failure check no
  t possible: ply strengths missing (Xt, Xc, Yt, Yc, S): give them on the command line or use a catalog row wi
  th all five)
  [satisfied    ] abs(tip_uz) <= 100: abs(tip_uz) <= 100.0  value 5.56263, limit 100, g -94.4374, g_rel -0.944
  374
FEASIBLE: unknown (constraints not evaluated)
  note: no density for material 'user ply': give --rho [kg/m^3]
  note: tip = free end xL: mean of the 11 edge nodes at xL
  note: failure check not possible: ply strengths missing (Xt, Xc, Yt, Yc, S): give them on the command line o
  r use a catalog row with all five
saved D:\Users\tvenkata\AppData\Local\Temp\1\oc_test\p.json
=== plate with rho and strengths
... (8 more lines omitted)
```

Tool: Bash - See which docs prior skill commit touched; read orchestrator header

```
cd "D:/Users/tvenkata/projects/ME295_project" && git show --stat 6ad47b8 | head -30; sed -n 1,58p .claude/agen
  ts/ThinShellOrchestrator.md
```

Result:

```
commit 6ad47b88ac4179de5726d3a289576259e23759d6
Author: TaraVenkatadri <tara.venkatadri@gmail.com>
Date:   Wed Nov 4 12:45:44 2026 -0800

    opensees-to-abaqus skill, results comparator agent, revised system diagram

    - New skill .claude/skills/opensees-to-abaqus: Abaqus/CAE script from the
      assembled OpenSeesPy script (shell-assemble-run), via abaqus-export;
      reference values from the script's own results; model check of the Abaqus
      script (stand-in API) against the script's recorded ops calls. Replaces the
      OpenSeesPyToAbaqusPythonGenerator agent (removed; v1 kept in agents_v1.0).
    - case_to_abaqus.py: generate() split into generate() + write_script();
      field output now also requests E and SE (section strains, curvatures).
    - New agent OpenSeesAbaqusResultsComparator: compares OpenSees results with
      an Abaqus ODB from the outputs only, with its own tools in
      .claude/verification (odb_extract.py for abaqus python, compare_results.py,
      stand-in ODB and tests: 25/25).
    - ThinShellOrchestrator: uses shell-assemble-run + opensees-to-abaqus and
      hands only results to the comparator.
    - Regression T22 (opensees-to-abaqus); default suite 257/257.
    - 11-4-26 (Tara): Abaqus scripts of the four cases; revised system diagram
      (system_diagram.py, system_diagram_verification.png).

    Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>

 .claude/REORGANIZATION_LOG_v2.0.md                 |    64 +
 .claude/agents/OpenSeesAbaqusResultsComparator.md  |   104 +
 .../agents/OpenSeesPyToAbaqusPythonGenerator.md    |    64 -
 .claude/agents/ThinShellOrchestrator.md            |    20 +-
 .../agents_v2.0/OpenSeesAbaqusResultsComparator.md |   104 +
---
name: ThinShellOrchestrator
description: Sets up, runs, checks and reports OpenSeesPy simulations of thin composite (or isotropic) shells
  in the ME295 project - flat plates, tape springs, TRAC/omega-type extruded sections - by running the project
   skills in order - laminate-properties (catalog lamina -> OpenSees laminate), shell-mesh (cross-section -> m
  eshed component), shell-bc-load (supports, load combination, solve), shell-visualize (curvature and stress p
  ictures) and failure-check (ply stresses and Hashin 2D index: how close to failure), and on request shell-as
  semble-run (one standalone OpenSeesPy script) and opensees-to-abaqus (equivalent Abaqus/CAE script of that m
  odel). Use for any shell simulation request, from material choice to failure check.
tools: Read, Write, Edit, Bash, Glob, Grep
... (55 more lines omitted)
```

Tool: Write

```
D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\SKILL.md
---
name: objective-constraints
description: Evaluates one design of an optimization from ONLY the results .npz written by the OpenSeesPy scri
  pt that shell-assemble-run assembles and runs - the objective (always: minimize the mass of the structure) a
  nd the constraints (always: no failure, governing Hashin 2D index from the failure-check skill < 1; plus any
   user constraints the orchestrator turns from the user's words into formulas, e.g. fixed or minimum length,
```

```
  maximum laminate thickness, maximum tip deflection, minimum stiffness, minimum buckling load) - and returns
  the mass, each constraint's value, margin g <= 0 and status, and feasibility as text and JSON for the orches
  trator. Use after every optimization run.
---

# objective-constraints

Script: `objective_constraints.py` (this folder; imports `../laminate-properties`,
`../shell-bc-load`, `../failure-check`). Units: N, mm, MPa; mass in kg.
Python: conda env `shells`: Federico `D:\Users\fbenazzo\miniconda3\envs\shells\python.exe`,
Tara `D:\Users\tvenkata\miniforge3\envs\shells\python.exe`.

**Input is only the results .npz** of the assembled script
(`<case dir>/results/<name>.npz`; shell_bc_load.py writes the same file). The
laminate, the case (geometry, supports, loads, analysis), the mesh and the
displacements and strains are all read from it. Do not give it the case
file, the script or the code pieces.

```
python objective_constraints.py <dir>/results/<case>.npz [--constraints <dir>/constraints.json]
       [--con "name: expr op limit" ...] [--value NAME=VALUE ...] [--rho KG_M3] [--json out.json]
       [failure-check options: --criterion --S23 --at --Xt --Xc --Yt --Yc --S --yield
        --boom ID --web S0 S1 --web-stack --flatten-radius]
python objective_constraints.py <dir>/results/<case>.npz --list   # every quantity, value, units
```
Writes `<results dir>/<case>_objcon.json` (or `--json`) and prints a summary.

## Objective: minimize mass (always)
m = sum over facets of (undeformed facet area) x (areal density sum rho_k t_k).
- Web facets of a TRAC/OM/CO section are two bonded flange laminates, so twice
  the areal density; they are found exactly as failure-check finds them
  (`--web`, case geometry `"web"`, or the catalog boom `--boom` / geometry `"boom"`).
- Density: the laminate's rho if > 0, else the lamina catalog row the
  material is named after (`L024 ...` from `--catalog`), else `--rho` [kg/m^3].
  The laminate pieces are written with rho = 0 (rho has no effect on a static
  run), so the catalog is the usual source; for user-defined plies or
  isotropic shells, ask for the density and pass `--rho`. Without a density
  the mass is "n/a": say so, never guess.
- Rigid end fixtures have no mass; the shell is the whole structure.

## Constraint always included: no failure
`F_governing < 1`: the larger of the simulation index (Hashin 2D by default,
max over facets, Gauss points, plies, 3 points per ply, web with the doubled
layup) and the coiling (flattening) index, with failure_check.py's own
functions, so the value equals failure_check.py's `GOVERNING F` line with
the same options. Missing strengths make it "not evaluated": ask for them.

## Turning the user's constraints into formulas (orchestrator's job)
Write one entry per constraint, keeping the user's words in `text`:
```json
{"constraints": [
 {"name": "tip deflection", "text": "tip may not deflect more than L/100", "expr": "abs(tip_uz)", "op": "<=",
  "limit": "L/100", "units": "mm"},
... (76 more lines omitted)
```

Result:

```
File created successfully at: D:\Users\tvenkata\projects\ME295_project\.claude\skills\objective-constraints\SK
  ILL.md (file state is current in your context — no need to Read it back)
```

Tool: Bash - Add the skill to README and orchestrator agent docs

```
cd "D:/Users/tvenkata/projects/ME295_project/.claude" && /d/Users/tvenkata/miniforge3/envs/shells/python.exe -
   <<'EOF'
from pathlib import Path
p = Path('skills/README.md'); s = p.read_text(encoding='utf-8')
s = s.replace("""                                                                                    |
                                                                                  +--> opensees-to-abaqus""",
"""                                                                                  |
                                                                                  +--> objective-constraints
  (mass; F_governing < 1 + user constraints -> g, feasible; JSON)
                                                                                  +--> opensees-to-abaqus""",
   1)
s = s.replace("""| shell-buckling |""", """| objective-constraints | new (11/4/26): objective/constraint calcu
  lator of the optimization loop; reads only the results .npz of the assembled script: mass (objective), gover
  ning Hashin F < 1 (via failure-check) and the user's constraints as formulas |
| shell-buckling |""", 1)
p.write_text(s, encoding='utf-8')
for f in ('agents/ThinShellOrchestrator.md', 'agents_v2.0/ThinShellOrchestrator.md'):
    p = Path(f); s = p.read_text(encoding='utf-8')
    old = """report its findings.
"""
    new = """report its findings.
```
```

```
In an optimization (minimize mass subject to constraints), turn each of the
user's constraints into a formula once, at the start, in a constraints JSON in
the study folder (objective-constraints SKILL.md: quantity names, which one
matches the user's words; ask if the wording is ambiguous), and after every
run evaluate the design with the objective-constraints skill on ONLY the
results .npz: `objective-constraints/objective_constraints.py <dir>/results/<case>.npz
--constraints <dir>/constraints.json`. It always adds the no-failure
constraint (governing Hashin F < 1) and returns the mass, each constraint's
g and status and feasibility in `<case>_objcon.json`; use that to choose the
next design. Quantities it cannot get from a static run (buckling load) come
from the skill that computes them, passed with `--value NAME=VALUE`.
"""
    assert s.count(old) == 1, f
    p.write_text(s.replace(old, new), encoding='utf-8')
EOF
git -C .. diff --stat
```

Result:

```
.claude/agents/ThinShellOrchestrator.md      | 12 ++++++++++
 .claude/agents_v2.0/ThinShellOrchestrator.md | 12 ++++++++++
 .claude/skills/README.md                     |  2 ++
 3 files changed, 26 insertions(+)
Shell cwd was reset to D:\Users\tvenkata\projects\ME295_project\Code for Project\11-4-26 (Tara)
```

Re-running all four result files after the edits, and leaving one worked example (constraint file + output JSON) in the 11-4-26 folder.

Tool: Bash - Run worked example and recheck other cases

```
cd "D:/Users/tvenkata/projects/ME295_project/Code for Project/11-4-26 (Tara)" && PY=/d/Users/tvenkata/miniforg
  e3/envs/shells/python.exe; OC=../../.claude/skills/objective-constraints/objective_constraints.py
cat > constraints_example.json <<'EOF'
{"constraints": [
 {"name": "tip deflection", "text": "the tip may not deflect more than L/50", "expr": "abs(tip_uz)", "op": "<=
  ", "limit": "L/50", "units": "mm"},
 {"name": "boom length", "text": "the boom is 500 mm long", "expr": "L", "op": "==", "limit": 500, "tol": 0.01
  , "units": "mm"},
 {"name": "thickness", "text": "laminate no thicker than 0.5 mm", "expr": "h_max", "op": "<=", "limit": 0.5, "
  units": "mm"},
 {"name": "tip stiffness", "text": "tip bending stiffness at least 1 N/mm", "expr": "abs(force_z) / abs(tip_uz
  )", "op": ">=", "limit": 1, "units": "N/mm"}
]}
EOF
$PY $OC results/tape_spring_L024_tip_load.npz --constraints constraints_example.json
for c in trac_like_combination trac_like_load_control; do $PY $OC results/$c.npz --json "$TEMP/oc_test/$c.json
  " | sed -n 2,4p; done
```

Result:

```
Objective / constraints: tape_spring_L024_tip_load  [[0/90/90/0] h = 0.32 mm]
OBJECTIVE  minimize mass: 0.0255893 kg (25.59 g)
CONSTRAINTS (g <= 0 is satisfied; g_rel = g / |limit|)
  [satisfied    ] no failure: F_governing < 1.0  value 0.106708, limit 1, g -0.893292, g_rel -0.893292
  [satisfied    ] tip deflection: abs(tip_uz) <= L/50  value 6.46024, limit 10, g -3.53976, g_rel -0.353976
  [satisfied    ] boom length: L == 500  value 500, limit 500, g -0.01, g_rel -2e-05
  [satisfied    ] thickness: h_max <= 0.5  value 0.32, limit 0.5, g -0.18, g_rel -0.36
  [satisfied    ] tip stiffness: abs(force_z) / abs(tip_uz) >= 1  value 1.54793, limit 1, g -0.54793, g_rel -0
  .54793
FEASIBLE: yes
  note: catalog L024 rho = 1600 kg/m^3
  note: tip = free end xL: rigid end master node at xL
  note: S23 = Yc/2 (default)
  note: coiling: flange flattened from r = 50 mm (smallest arc radius of the case cross-section), F = 0.1067 (
  matrix tension)
  note: critical point of the simulation: {'group': 'flange', 'facet': 1900, 'i': 0, 'j': 19, 'ply': 2, 'theta
  ': 90.0, 'at': 'bottom', 'mode': 'matrix tension', 's1_s2_t12': [1.487726808555989, 7.321080605804418, 0.951
  4952755376491]}
saved results\tape_spring_L024_tip_load_objcon.json
OBJECTIVE  minimize mass: 0.00562534 kg (5.625 g)
CONSTRAINTS (g <= 0 is satisfied; g_rel = g / |limit|)
  [satisfied    ] no failure: F_governing < 1.0  value 0.538646, limit 1, g -0.461354, g_rel -0.461354
OBJECTIVE  minimize mass: 0.00562534 kg (5.625 g)
CONSTRAINTS (g <= 0 is satisfied; g_rel = g / |limit|)
  [satisfied    ] no failure: F_governing < 1.0  value 0.335368, limit 1, g -0.664632, g_rel -0.664632
```

Tool: wrap_up

```
{
 "recap": "The objective-constraints skill is built and tested on the four 11-4-26 results files; its Hashin v
  alues match failure-check exactly. The README and the orchestrator agent now point to it. Nothing is committ
  ed yet.",
 "ask": "Commit the new skill and doc updates?",
 "actions": [
```

```
      {
       "action": "commit"
      },
      {
       "label": "Not yet"
      }
     ]
    }
```

Result:

```
Shown in the next-step box. Your closing message is your answer to the user: write it now, in full, as the las
  t thing you say this turn. The box asks your question, so do not ask it again.
```

I've built the objective/constraint calculator as a new skill, `objective-constraints`, and tested it on all four results files in `11-4-26 (Tara)/results`. Its failure index matches failure-check's `GOVERNING F` exactly in every case, and the masses agree with hand calculations.

What it reads: only the results `.npz` that the assembled OpenSeesPy script writes. The laminate, geometry, loads, analysis type, displacements and strains all come from that file. Two things can be passed in on the command line:

- `--rho`, when the density can't be found automatically;
- `--value NAME=VALUE`, for numbers a static run can't produce (see buckling below).

Objective (always minimize mass): mass is each undeformed element's area times its laminate's mass per unit area, summed over the model.

- TRAC/omega web elements count twice, because a web is two bonded laminates. They are identified the same way failure-check identifies them.
- **Density:** the laminate pieces are written with `rho = 0`, so density comes from the lamina catalog row the material is named after (L024 → 1600 kg/m³). For user-defined plies, pass `--rho`. With no density available, mass is reported as "n/a" rather than guessed.
- **Example:** the tape spring comes out at 25.59 g, matching 500 × 100 mm × 0.512 kg/m².

Failure constraint (always included): `F_governing < 1`, computed with failure-check's own functions. It takes the larger of the simulation's Hashin index and the coiling (flattening) index.

Your other constraints: the orchestrator turns each one, written in natural language, into an entry in a JSON file, e.g. `"expr": "abs(tip_uz)", "op": "<=", "limit": "L/100"`. Inline `--con "name: expr <= limit"` also works.

- **Writing formulas:** `--list` prints all ~45 available quantities: mass, length, section width/height, thickness, radii, tip and maximum displacements, applied forces, load factors and failure indices. `SKILL.md` has a guide for mapping wording to quantities, e.g. "stiffness" becomes `abs(force_z)/abs(tip_uz)`.
- **Safety:** formulas can only use those quantities, numbers, arithmetic and a short list of math functions. Unknown names and code injection are refused.
- **Result format:** every constraint is reported as g ≤ 0, along with g/|limit| and a status of `satisfied`, `VIOLATED` or `not evaluated: <reason>`.
- **Feasibility:** `feasible` is true, false, or unknown when something couldn't be evaluated.
- **Output file:** everything goes to `<case>_objcon.json` for the orchestrator.

Buckling: a buckling load isn't in a static results file, so taking only the aggregator's output can't produce it. Such a constraint is reported "not evaluated" unless the orchestrator runs shell-buckling and passes `--value buckling_load=…`. Nonlinear runs do report the peak load factor and whether a limit point was passed.

Files:

- New: objective_constraints.py and SKILL.md.
- Updated: the skills README.md, and `ThinShellOrchestrator.md` in both `agents/` and `agents_v2.0/`, which now tells the orchestrator how to write the constraint file and when to call the skill.
- Worked example: constraints_example.json and its output, `results/tape_spring_L024_tip_load_objcon.json`.

I didn't change any existing skill scripts, so I didn't rerun the regression suite. There's also no regression test for the new skill yet. The system diagram already has this box, so I left it unchanged.

User

Create a PDF of this conversation and put it in the Claude Conversations folder. This is Tara conversation 3 today. Put a brief description of the conversation in the file title

Claude

Tool: Bash - Find conversations folder and existing PDFs

```
cd "D:/Users/tvenkata/projects/ME295_project" && find . -maxdepth 3 -type d -iname "*conversation*"; find . -m
  axdepth 4 -iname "*conversation*.pdf" | sort | tail -15
```

Result:

```
./Claude Conversations
./Claude Conversations/Tara 10-23-26 Conversation 1 (Python setup and Federico's 10-22 test cases).pdf
./Claude Conversations/Tara 10-23-26 Conversation 2 (Building material property agents).pdf
./Claude Conversations/Tara 10-23-26 Conversation 3 (Testing material property agents).pdf
./Claude Conversations/Tara 10-26-26 Conversation 1 (CFRP laminate test cases, result viewer and Abaqus agents
  ).pdf
./Claude Conversations/Tara 11-4-26 Conversation 1 (Skill to join laminate, mesh and BC-load OpenSeesPy code i
  nto one script, GitHub sync).pdf
./Claude Conversations/Tara 11-4-26 Conversation 2 (OpenSees-to-Abaqus skill, results comparator agent and sys
  tem diagram).pdf
Shell cwd was reset to D:\Users\tvenkata\projects\ME295_project\Code for Project\11-4-26 (Tara)
```

Tool: Bash - Locate session transcript and check reportlab

```
ls -la /d/Users/tvenkata/.claude/projects/ | head; find /d/Users/tvenkata/.claude/projects -name "e44cd040*" 2
  >/dev/null; /d/Users/tvenkata/miniforge3/envs/shells/python.exe -c "import reportlab; print(reportlab.__vers
  ion__)"; python3 -c "import reportlab" 2>&1
```

Result:

```
Exit code 127
total 20
drwxr-xr-x 1 tvenkata 1049089 0 Nov  4 11:56 .
drwxr-xr-x 1 tvenkata 1049089 0 Nov  4 13:41 ..
drwxr-xr-x 1 tvenkata 1049089 0 Oct 12 15:52 D--Users-tvenkata-projects
drwxr-xr-x 1 tvenkata 1049089 0 Oct 15 20:58 D--Users-tvenkata-projects-ME295-project
drwxr-xr-x 1 tvenkata 1049089 0 Oct 12 16:18 D--Users-tvenkata-projects-ME295-project-Code-for-Project
drwxr-xr-x 1 tvenkata 1049089 0 Oct 23 13:43 D--Users-tvenkata-projects-ME295-project-Code-for-Project-10-23-2
  6--Tara-
drwxr-xr-x 1 tvenkata 1049089 0 Nov  4 11:16 D--Users-tvenkata-projects-ME295-project-Code-for-Project-10-26-2
  6--Federico-and-Tara-
drwxr-xr-x 1 tvenkata 1049089 0 Nov  4 13:41 D--Users-tvenkata-projects-ME295-project-Code-for-Project-11-4-26
  --Tara-
/d/Users/tvenkata/.claude/projects/D--Users-tvenkata-projects-ME295-project-Code-for-Project-11-4-26--Tara-/e4
  4cd040-9b42-4dd2-823d-4414f8843279
/d/Users/tvenkata/.claude/projects/D--Users-tvenkata-projects-ME295-project-Code-for-Project-11-4-26--Tara-/e4
  4cd040-9b42-4dd2-823d-4414f8843279.jsonl
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'reportlab'
/usr/bin/bash: line 1: python3: command not found
```

Tool: Bash - Check PDF libraries and transcript format

```
ls /d/Users/tvenkata/miniforge3/envs; for p in /d/Users/tvenkata/miniforge3/python.exe /d/Users/tvenkata/minif
  orge3/envs/*/python.exe; do echo "$p: $($p -c 'import importlib.util as u; print([m for m in ("reportlab","f
  pdf","pypdf","fitz","matplotlib") if u.find_spec(m)])' 2>&1)"; done; J=/d/Users/tvenkata/.claude/projects/D-
  -Users-tvenkata-projects-ME295-project-Code-for-Project-11-4-26--Tara-/e44cd040-9b42-4dd2-823d-4414f8843279.
  jsonl; wc -l $J; head -c 600 $J
```

Result:

```
pdftools
shells
/d/Users/tvenkata/miniforge3/python.exe: []
/d/Users/tvenkata/miniforge3/envs/pdftools/python.exe: ['reportlab']
/d/Users/tvenkata/miniforge3/envs/shells/python.exe: ['matplotlib']
221 /d/Users/tvenkata/.claude/projects/D--Users-tvenkata-projects-ME295-project-Code-for-Project-11-4-26--Tara
  -/e44cd040-9b42-4dd2-823d-4414f8843279.jsonl
{"type":"queue-operation","operation":"enqueue","timestamp":"2026-11-04T21:41:40.709Z","sessionId":"e44cd040-9
  b42-4dd2-823d-4414f8843279","content":"I want to build the objective/constraint calculator skill in the syst
  em diagram in the 11-4-26 (Tara) folder.  The objective will always be to minimize mass of the structure.  T
  he constraints will always include a check against the failure criteria using the failure-check skill.  Ther
  e may also be other constraints, including but not limited to dimensions along certain axes (e.g. fixed or m
  inimum length for a boom or maximum thickness for the lami
```

Tool: Bash - Find prior PDF script and summarize transcript records

```
cd /d/Users/tvenkata; grep -rl "reportlab" --include=*.py projects/ME295_project /d/Users/tvenkata/AppData/Loc
```

```
  al/Temp/1/claude 2>/dev/null | head; J=/d/Users/tvenkata/.claude/projects/D--Users-tvenkata-projects-ME295-p
  roject-Code-for-Project-11-4-26--Tara-/e44cd040-9b42-4dd2-823d-4414f8843279.jsonl; /d/Users/tvenkata/minifor
  ge3/envs/shells/python.exe -c "
import json,collections
c=collections.Counter()
for l in open(r'$J',encoding='utf-8'):
    d=json.loads(l); m=d.get('message',{});
    t=[b.get('type') for b in m.get('content',[])] if isinstance(m.get('content'),list) else [type(m.get('cont
  ent')).__name__]
    c[(d['type'],tuple(t))]+=1
for k,v in c.items(): print(v,k)
"
```

Result:

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 4, in <module>
FileNotFoundError: [Errno 2] No such file or directory: '/d/Users/tvenkata/.claude/projects/D--Users-tvenkata-
  projects-ME295-project-Code-for-Project-11-4-26--Tara-/e44cd040-9b42-4dd2-823d-4414f8843279.jsonl'
```

Tool: Bash - Find prior PDF script and summarize transcript records

```
grep -rl "reportlab" --include=*.py /d/Users/tvenkata/projects/ME295_project /d/Users/tvenkata/AppData/Local/T
  emp/1/claude 2>/dev/null | head; /d/Users/tvenkata/miniforge3/envs/shells/python.exe -c "
import json,collections
J=r'D:\Users\tvenkata\.claude\projects\D--Users-tvenkata-projects-ME295-project-Code-for-Project-11-4-26--Tara
  -\e44cd040-9b42-4dd2-823d-4414f8843279.jsonl'
c=collections.Counter()
for l in open(J,encoding='utf-8'):
    d=json.loads(l); m=d.get('message',{}) or {}
    t=[b.get('type') for b in m.get('content',[])] if isinstance(m.get('content'),list) else [type(m.get('cont
  ent')).__name__]
    c[(d['type'],tuple(t))]+=1
for k,v in c.items(): print(v,k)
"
```

Result:

```
4 ('queue-operation', ('NoneType',))
2 ('user', ('str',))
15 ('attachment', ('NoneType',))
18 ('last-prompt', ('NoneType',))
19 ('custom-title', ('NoneType',))
2 ('file-history-snapshot', ('NoneType',))
18 ('atis-latch', ('NoneType',))
18 ('agent-name', ('NoneType',))
1 ('api-request-shape', ('NoneType',))
1 ('api-request-blob', ('text', 'text', 'text'))
2 ('api-request-blob', ('text',))
26 ('api-request', ('NoneType',))
23 ('assistant', ('thinking',))
32 ('assistant', ('tool_use',))
31 ('user', ('tool_result',))
2 ('api-request-blob', ('thinking', 'tool_use', 'tool_use'))
4 ('api-request-blob', ('tool_result', 'tool_result'))
4 ('assistant', ('text',))
1 ('api-request-blob', ('tool_use', 'tool_use'))
2 ('file-history-delta', ('NoneType',))
1 ('api-request-blob', ('thinking', 'text', 'tool_use', 'tool_use'))
1 ('api-request-blob', ('thinking', 'thinking', 'tool_use', 'tool_use', 'tool_use', 'tool_use'))
1 ('system', ('NoneType',))
```

Building the PDF from this session's transcript with reportlab (in your `pdftools` env); thinking blocks are left out and long tool outputs are truncated.

Tool: Write

```
D:\Users\tvenkata\AppData\Local\Temp\1\claude\D--Users-tvenkata-projects-ME295-project-Code-for-Project-11-4-2
  6--Tara-\e44cd040-9b42-4dd2-823d-4414f8843279\scratchpad\transcript_pdf.py
"""Session transcript (.jsonl) -> PDF: user messages, assistant text, tool calls
and (truncated) tool results; thinking omitted.
  python transcript_pdf.py session.jsonl out.pdf "Title"
"""
import json
import re
import sys
from xml.sax.saxutils import escape

from reportlab.lib import colors
from reportlab.lib.enums import TA_LEFT
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import inch
```

```
from reportlab.platypus import Paragraph, Preformatted, SimpleDocTemplate, Spacer

src, out, title = sys.argv[1:4]
MAX_LINES, MAX_CHARS = 40, 110
SKIP_TOOLS = {'mcp__ccd_turn__update_plan'}

ss = getSampleStyleSheet()
H = ParagraphStyle('h', parent=ss['Heading2'], spaceBefore=10, spaceAfter=4)
body = ParagraphStyle('b', parent=ss['BodyText'], fontSize=9.5, leading=12.5, alignment=TA_LEFT)
user_h = ParagraphStyle('u', parent=H, textColor=colors.HexColor('#6a2b9a'))
asst_h = ParagraphStyle('a', parent=H, textColor=colors.HexColor('#1f5fbf'))
tool_h = ParagraphStyle('t', parent=ss['BodyText'], fontSize=8.5, textColor=colors.HexColor('#444444'),
                        spaceBefore=4)
mono = ParagraphStyle('m', parent=ss['Code'], fontSize=7, leading=8.4, backColor=colors.HexColor('#f4f4f4'),
                      borderPadding=3, spaceAfter=4)


def wrap(text, n=MAX_CHARS, max_lines=MAX_LINES):
    lines = []
    for ln in text.splitlines():
        while len(ln) > n:
            lines.append(ln[:n])
            ln = '  ' + ln[n:]
        lines.append(ln)
    if len(lines) > max_lines:
        lines = lines[:max_lines] + [f'... ({len(lines) - max_lines} more lines omitted)']
    return '\n'.join(lines)


def md(text):
    """Light markdown -> reportlab paragraphs (bold, code, links, bullets)."""
    out = []
    for block in re.split(r'\n{2,}', text.strip()):
        if block.startswith('```'):
            out.append(Preformatted(wrap(block.strip('`').split('\n', 1)[-1]), mono))
            continue
        lines = []
        for ln in block.splitlines():
            ln = escape(ln)
            ln = re.sub(r'\*\*(.+?)\*\*', r'<b>\1</b>', ln)
            ln = re.sub(r'`([^`]+)`', r'<font face="Courier">\1</font>', ln)
            ln = re.sub(r'\[([^\]]+)\]\([^)]+\)', r'\1', ln)
            ln = re.sub(r'^(\s*)[-*] ', lambda m: ' ' * (2 * len(m[1]) + 2) + '&bull; ', ln)
            ln = re.sub(r'^#+ ', '', ln)
... (60 more lines omitted)
```

Result:

```
File created successfully at: D:\Users\tvenkata\AppData\Local\Temp\1\claude\D--Users-tvenkata-projects-ME295-p
  roject-Code-for-Project-11-4-26--Tara-\e44cd040-9b42-4dd2-823d-4414f8843279\scratchpad\transcript_pdf.py (fi
  le state is current in your context — no need to Read it back)
```

Tool: Bash - Generate the conversation PDF

```
S='D:\Users\tvenkata\AppData\Local\Temp\1\claude\D--Users-tvenkata-projects-ME295-project-Code-for-Project-11-
  4-26--Tara-\e44cd040-9b42-4dd2-823d-4414f8843279\scratchpad'; J='D:\Users\tvenkata\.claude\projects\D--Users
  -tvenkata-projects-ME295-project-Code-for-Project-11-4-26--Tara-\e44cd040-9b42-4dd2-823d-4414f8843279.jsonl'
OUT='D:\Users\tvenkata\projects\ME295_project\Claude Conversations\Tara 11-4-26 Conversation 3 (Objective-cons
  traint calculator skill - mass objective, failure and user constraints).pdf'
/d/Users/tvenkata/miniforge3/envs/pdftools/python.exe -I "$S\transcript_pdf.py" "$J" "$OUT" "Tara 11-4-26 Conv
  ersation 3: Objective/constraint calculator skill" && ls -la "/d/Users/tvenkata/projects/ME295_project/Claud
  e Conversations/"
```